

Two-Phase Commit Protocol

Bucă Mihnea-Vicențiu Luculescu Teodor



Facultatea de Matematică și Informatică
Universitatea din București

24 Iunie 2026

Structura prezentării

Protocolul

Two-Phase Commit ca protocol de *atomic transaction commit*.

Modelul formal

Specificația TLA⁺, invarianții verificați și rezultatele TLC.

Implementări

Implementări concurente în Java și Go, cu modele de execuție diferite.

Identificarea erorilor

Erorile analizate cu TLC, JUnit, JavaPathFinder și execuții Go.

Problema: commit atomic

Scop

O tranzacție distribuită trebuie să se încheie cu o singură decizie globală.

- toți participanții confirmă tranzacția (COMMIT); sau
- toți participanții o anulează (ABORT).

Safety property

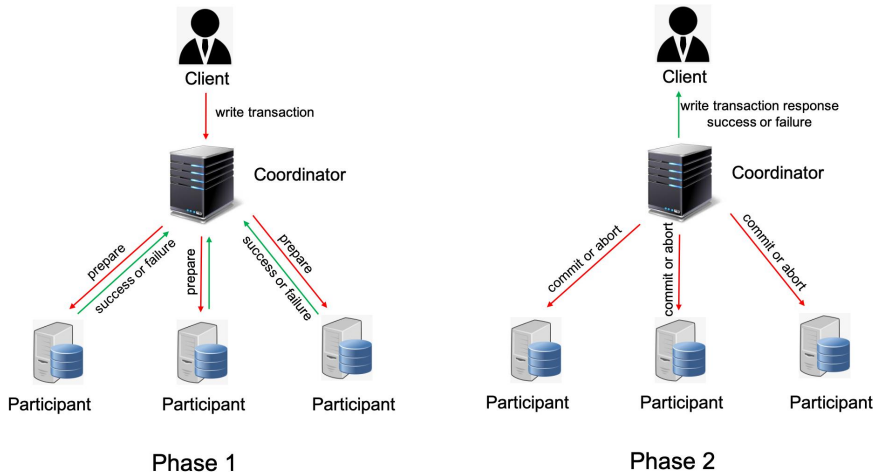
Nicio execuție nu trebuie să lase un participant în starea COMMITTED și altul în starea ABORTED.

$$\neg \exists p, q : committed(p) \wedge aborted(q)$$

Ideea centrală

Protocolul urmărește rezultatele corecte în cazuri normale și excluderea tuturor stărilor finale inconsistente.

Cele două faze ale protocolului



Faza 1 colectează voturile; faza 2 distribuie decizia globală COMMIT sau ABORT.

Regula de bază

Regula corectă

Commit-ul este permis numai dacă toți participanții au ajuns în starea PREPARED.

```
tmPrepared = RM
```

Slăbirea restricțiilor

Efectuarea commit-ului după un singur vot pozitiv nu mai respectă protocolul Two-Phase Commit.

```
tmPrepared # {}
```

Verificarea compară condiția validă cu condiția incorectă 'există cel puțin un vot pozitiv'.

Modelul formal: ce este reprezentat?

Variabilă / acțiune	Semnificație
rmState[rm]	starea participantului: working, prepared, committed, aborted
tmState	starea coordonatorului: init, committed, aborted
tmPrepared	mulțimea participanților al căror vot prepared a fost înregistrat
msgs	mesajele trimise în model
RMPprepare, TMCommit, TMAbort	tranzițiile principale ale protocolului

Invarianti verificați

TPTypeOK verifică tipurile și domeniile variabilelor, iar TPConsistent verifică acordul atomic între participanți.

TLC: modelul corect

```
~/Projects/transaction_commit main !2 ? 0.02s mvb@arch-linux
) java -XX:+UseParallelGC -cp tools/tla2tools.jar tlc2.TLC -config tla/TwoPhase.cfg tla/TwoPhase.tla
TLC2 Version 2.19 of 08 August 2024 (rev: 5a47802)
Running breadth-first search Model-Checking with fp 91 and seed -6041324745753915070 with 1 worker on 24 cores with 3378MB heap and 64MB offheap m
emory [pid: 25387] (Linux 7.0.5-arch1-1-g14 amd64, Arch Linux 17.0.19 x86_64, MSBDiskFPSets, DiskStateQueue).
Parsing file /home/mvb/Projects/transaction_commit/tla/TwoPhase.tla
Parsing file /home/mvb/Projects/transaction_commit/tla/TCommit.tla
Semantic processing of module TCommit
Semantic processing of module TwoPhase
Starting... (2026-06-15 14:52:25)
Computing initial states...
Finished computing initial states: 1 distinct state generated at 2026-06-15 14:52:25.
Model checking completed. No error has been found.
  Estimates of the probability that TLC did not check all reachable states
    because two distinct states had the same fingerprint:
      calculated (optimistic): val = 1.3E-14
1146 states generated, 288 distinct states found, 0 states left on queue.
The depth of the complete state graph search is 11.
The average outdegree of the complete state graph is 1 (minimum is 0, the maximum 7 and the 95th percentile is 3).
Finished in 00s at (2026-06-15 14:52:25)
```

Rezultat

TLC finalizează verificarea fără încălcarea invariantilor.

- 3 resource managers
- 288 stări distincte
- 0 încălcări ale proprietății de consistență

Explorarea spațiului de stări cu TLC

```
1146 states generated, 288 distinct states found, 0 states left on queue.  
The depth of the complete state graph search is 11.  
The average outdegree of the complete state graph is 1 (minimum is 0, the maximum 7 and the 95th percentile is 3).  
Finished in 00s at (2026-06-15 14:52:25)
```

Ce înseamnă 1.146 stări generate?

Mai multe ordini de execuție pot ajunge la aceeași stare globală.

Ce înseamnă 288 stări distincte?

TLC păstrează fiecare stare distinctă o singură dată și continuă explorarea exhaustivă.

Eroare detectată de TLC

```
1 java -XX:+UseParallelGC -cp tools/tla2tools.jar tlc2.TLC -config tla/TwoPhaseBuggy.cfg tla/TwoPhaseBuggy.tla
TLC2 Version 2.19 of 08 August 2024 (rev: 3a47802)
Running breadth-first search Model-Checking with fg 57 and seed -681215531a231a79a1 with 1 worker on 24 cores with 33
78MB heap and 64MB offheap memory [pid: 25444] (Linux 7.0.5-arch1-1-g14 amd64, Arch Linux 17.0.19 x86_64, MSBDisKFSPE
T, DiskStateQueue).
Parsing file /home/mvb/Projects/transaction_commit/tla/TwoPhaseBuggy.tla
Parsing file /home/mvb/Projects/transaction_commit/tla/TCommit.tla
Semantic processing of module TCommit
Semantic processing of module TwoPhaseBuggy
Starting... (2026-06-23 19:49:11)
Computing initial states...
Finished computing initial states: 1 distinct state generated at 2026-06-23 19:49:11.
Error: Invariant TPConsistent is violated.
Error: The behavior up to this point is:
State 1: <initial predicate>
/\ msgs = {}
/\ rmState = {r1 :-> "working" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "init"
/\ tmPrepared = {}

State 2: <RMPrep line 40, col 3 to line 49, col 38 of module TwoPhaseBuggy>
/\ msgs = {ltype l-> "Prepared", rm l-> r1}
/\ rmState = {r1 :-> "prepared" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "init"
/\ tmPrepared = {}

State 3: <TMRCvPrep line 26, col 3 to line 29, col 41 of module TwoPhaseBuggy>
/\ msgs = {ltype l-> "Prepared", rm l-> r1}
/\ rmState = {r1 :-> "prepared" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "init"
/\ tmPrepared = {r1}

State 4: <TCommit line 32, col 3 to line 37, col 38 of module TwoPhaseBuggy>
/\ msgs = {ltype l-> "Commit", ltype l-> "Prepared", rm l-> r1}
/\ rmState = {r1 :-> "prepared" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "committed"
/\ tmPrepared = {r1}

State 5: <RMRCvCommitMsg line 57, col 3 to line 59, col 44 of module TwoPhaseBuggy>
/\ msgs = {ltype l-> "Commit", ltype l-> "Prepared", rm l-> r1}
/\ rmState = {r1 :-> "committed" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "committed"
/\ tmPrepared = {r1}

State 6: <RMChooseToAbort line 52, col 3 to line 54, col 44 of module TwoPhaseBuggy>
/\ msgs = {ltype l-> "Commit", ltype l-> "Prepared", rm l-> r1}
/\ rmState = {r1 :-> "committed" @ r2 :-> "aborted" @ r3 :-> "working"}
/\ tmState = "committed"
/\ tmPrepared = {r1}

457 states generated, 158 distinct states found, 66 states left on queue.
The depth of the complete state graph search is 6.
The average outdegree of the complete state graph is 2 (minimum is 0, the maximum 7 and the 95th percentile is 5).
Finished in 80s at (2026-06-23 19:49:11)
```

Contraexemplu

Un participant ajunge la COMMITTED, iar altul ajunge la ABORTED.

- 1 r1 intră în starea prepared
- 2 coordonatorul face commit prea devreme
- 3 un alt RM execută abort
- 4 se încalcă invariantul TPConsistent

Implementarea Java

Arhitectură

- Coordinator
- Participant
- Message
- NetworkSimulator
- FailureInjector

Model de concurență

Cererile de pregătire sunt trimise asincron prin `CompletableFuture` și un *executor service*.

Safety property

```
preparedParticipants.size()  
== participants.size()
```

Limitare

Implementarea este intenționat simplificată și rulează complet în memorie.

Legătura între TLA⁺ și Java

Element TLA ⁺	Reprezentare Java
rmState[rm]	câmpul sincronizat Participant.state
tmState	starea internă a clasei Coordinator
tmPrepared	mulțimea preparedParticipants
RMPprepare	metoda Participant.onPrepare
TMCommit/TMAbort	ramura de decizie din Coordinator.execute
TPConsistent	metoda TransactionResult.isConsistent()

Idee

Implementarea Java adaptează conceptele modelului formal la un program concurrent, păstrând regula conform căreia decizia de COMMIT este permisă numai după pregătirea tuturor participanților.

Testarea implementării Java cu JUnit

```
~/Projects/transaction_commit main 0.02s mvb@arch-Linux
$ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< pc.project:transaction-commit >-----
[INFO] Building transaction-commit 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ transaction-commit ---
[INFO] skip non existing resourceDirectory /home/mvb/Projects/transaction_commit/src/main/resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ transaction-commit ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ transaction-commit ---
[INFO] skip non existing resourceDirectory /home/mvb/Projects/transaction_commit/src/test/resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ transaction-commit ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ transaction-commit ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running TwoPhaseCommitTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.048 s -- in TwoPhaseCommitTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.947 s
[INFO] Finished at: 2024-06-15T15:10:34+03:00
[INFO] -----
```

Scenarii testate

- toți participanții votează PREPARED
- un participant votează ABORT
- participant indisponibil
- livrarea repetată a aceleiași decizii
- coordonator cu o condiție de commit eronată

JUnit: ce demonstrează testele?

Model corect

Testele confirmă cazurile reprezentative: commit global, abort global, participant indisponibil și decizii duplicate.

De ce testul este reușit?

Testul reușește deoarece verifică în mod explicit că utilizarea condiției de commit eronată conduce la un rezultat inconsistent.

Limitare

JUnit validează scenarii concrete, dar nu explorează exhaustiv toate intercalările posibile ale firelor de execuție.

JavaPathFinder: modelul corect

```
) tools/jpf-core/bin/jpf tests/JavaPathFinder/correct_two_phase.jpf
JavaPathfinder core system v8.0 (rev 2bf98722c49a8149f9a6f68acb9edc6ba5cdd557) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test
CorrectTwoPhaseJpf.main()

===== search started: 6/16/26, 6:25 PM

===== results
no errors detected

===== statistics
elapsed time:      00:00:00
states:           new=593,visited=683,backtracked=1276,end=32
search:           maxDepth=17,constraints=0
choice generators: thread=393 (signal=0,lock=11,sharedRef=180,threadApi=18,reschedule=184), data=200
heap:             new=1199,released=1335,maxLive=537,gcCycles=1076
instructions:     53066
max memory:       238MB
loaded code:      classes=89,methods=2423

===== search finished: 6/16/26, 6:25 PM
```

Modelul redus folosește fire de execuție și `Verify.getBoolean()` pentru a explora alegeri nedeterminate.

JavaPathFinder: modelul defect

```
> tools/jpf-core/bin/jpf tests/JavaPathFinder/buggy_two_phase.jpf
JavaPathFinder core system v8.0 (rev 2bf98722c49a8149f9a6f68acb9edc6ba5cdd557) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test
BuggyTwoPhaseJpf.main()

===== search started: 6/16/26, 5:55 PM

===== error 1
gov.nasa.jpf.vm.NoUncaughtExceptionsProperty
java.lang.AssertionError: Inconsistent decision for voteMask=1
    at BuggyTwoPhaseJpf.assertConsistent(BuggyTwoPhaseJpf.java:56)
    at BuggyTwoPhaseJpf.verifyTransaction(BuggyTwoPhaseJpf.java:39)
    at BuggyTwoPhaseJpf.main(BuggyTwoPhaseJpf.java:13)

===== snapshot #1
no live threads

===== results
error #1: gov.nasa.jpf.vm.NoUncaughtExceptionsProperty "java.lang.AssertionError: Inconsistent decision fo..."

===== statistics
elapsed time:      00:00:00
states:           new=1,visited=0,backtracked=0,end=1
search:           maxDepth=1,constraints=0
choice generators: thread=1 (signal=0,lock=1,sharedRef=0,threadApi=0,reschedule=0), data=0
heap:             new=549,released=0,maxLive=0,gcCycles=0
instructions:     10266
max memory:       238MB
loaded code:      classes=90,methods=2361

===== search finished: 6/16/26, 5:55 PM
```

Aserțiunea de consistență eșuează deoarece modelul permite COMMIT fără acordul tuturor participanților.

Implementarea Go

Modelul de concurență

- *goroutine* pentru transaction manager
- un *goroutine* pentru fiecare resource manager
- *channel* comun pentru voturi
- *channel* separat pentru decizia fiecărui participant

Varianta ernoată

Coordonatorul efectuează commit după primul vot PreparedVote primit.

Observație

Absența race-condition nu garantează respectarea protocolului Two-Phase Commit.

Program Go: *unbuffered channel*

```
$ go run TwoPhaseAlgorithmLogicBug.go --rm=0,0,1
rm3 prepare FAILED
rm2 prepared
TM received PREPARE_FAILED from rm3
rm1 prepared
TM received PREPARED from rm2
TM committing
fatal error: all goroutines are asleep - deadlock!
```

Dacă transaction manager-ul returnează prea devreme, alte *goroutine* pot rămâne blocate la operații de trimitere prin *channel*.

Program Go: *buffered channel*

```
$ go run TwoPhaseAlgorithmLogicBug.go --rm=0,0,1 --bufferedChannel
rm1 prepared
rm3 prepare FAILED
rm2 prepared
TM received PREPARED from rm1
TM committing
rm1 committed
rm2 committed

Final state:
TM = committed
rm1 = committed
rm2 = committed
rm3 = aborted
```

Cu *channel-uri buffered* se evită *deadlock-ul*, dar se observă participanți în stările *commit* și *abort*

Contribuția fiecărui instrument

TLC

Explorează exhaustiv modelul finit TLA⁺ și produce contraexemplu pentru bug.

Go runtime

Verifică existența condițiilor de *deadlock* pentru *goroutine* și *channel*-uri.

JUnit

Verifică scenarii reprezentative în implementarea Java completă.

Variante defecte

Demonstrează că procesul de verificare respinge o condiție de commit incorectă.

JavaPathFinder

Explorează alegeri nedeterminate și planificări ale firelor în modelul Java redus.

Concluzie

Fiecare instrument observă alt aspect: model formal, teste concrete, planificări sau execuții reale.

Dezvoltare asistată de AI

Utilizarea AI

Prompturile sunt organizate pe sarcini și documentează modul în care componentele Java și Go au fost generate, revizuite și verificate.

Rezultate

Comenzile și capturile de ecran documentează rezultatele obținute cu TLC, JUnit, JPF, execuția Go și Go runtime.

Aspect important

Codul generat cu ajutorul AI a fost acceptat numai după compilare, testare, verificare formală sau verificare în timpul execuției.

Aspecte neexplorate

- stocarea persistentă a stării protocolului
- recuperarea după defectarea proceselor
- comunicare printr-o rețea reală
- gestionarea identificatorilor de tranzacție

Concuzii în urma verificării

Rezultatele oferă dovezi solide pentru abstracțiile selectate, dar nu reprezintă o demonstrație de corectitudine pentru sisteme de producție și configurații arbitrare.

Concluzie

Concluzia principală

Coordonatorul trebuie să aștepte ca fiecare participant să ajungă în starea de pregătire înainte de a decide COMMIT.

De ce este important

Slăbirea condiției de commit permite apariția unor stări finale inconsistente.

Dovezi

TLC, JUnit, JavaPathFinder și experimentele Go evidențiază aceeași eroare de commit prematur din perspective diferite.

**Mulțumim pentru
atenție!**

Întrebări?